

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Overloading degli operatori

Stefano Ghidoni

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



Agenda

- Overloading degli operatori
  - Helper function
  - Funzioni membro
- Esempi



Reimplementare gli operatori

- Recall: espressioni e operatori
- Gli operatori sono elementi fondamentali nelle espressioni
- Operatori predefiniti per i tipi built-in
- Che succede agli UDT?
  - È possibile usare gli operatori?
  - Cosa significa sommare due date?
- Possibile se ne è definito il comportamento



Overloading degli operatori

- In C++ è possibile implementare quasi tutti gli operatori per operandi di tipo definito dall'utente (class o enum)
- **Operator overloading**
- Può essere effettuato usando una funzione
  - Helper function
  - Funzioni membro
- Overloading solo di operatori esistenti
  - Con lo stesso numero di operandi e la stessa sintassi

Overloading degli operatori

- L'overloading è implementato creando una funzione con un nome specifico
  - operator+, operator++, operator*, operator[], ...
- Il C++ riconosce questo pattern e traduce

```
Date d {2010, Mon::feb, 21};

++d;    // equivalente a operator++(d),
        // oppure a d.operator++()
```

Overloading degli operatori

- Deve essere presente almeno un argomento UDT
  - Non è possibile definire int operator+(int, int);
- Uno strumento potente, ma:
  - È importante non definire operatori con significati contro-intuitivi
  - È sensato definire gli operatori solo se hanno concettualmente senso per quel tipo di dato
- Gli operatori più utili: =, ==, !=, <, [], ()

Overloading degli operatori

- Ogni operatore ha il suo pattern
  - Il pattern è diverso se è funzione membro o funzione esterna:
    - Funzione membro: **il primo operando è l'oggetto della classe**
    - Helper function: i due operandi sono argomenti della funzione

Overloading: membro

- Operatori implementati con funzione membro
  - Un argomento in meno
- Il primo argomento è sostituito dall'oggetto su cui è invocata la funzione
  - Lo UDT deve essere il primo argomento
    - int + T non è accettato – lo è solo T + int
    - Meno flessibile!

Overloading degli operatori

- Ogni operatore ha il suo pattern
  - Il pattern è diverso se è funzione membro o funzione esterna

```
MyClass operator+(const MyClass&, const MyClass&); // Helper
// Oppure
MyClass MyClass::operator+(const MyClass&);        // Membro

// Esempio con un altro operatore:
vector operator+=(const Vector&, int);             // OK

int operator+(int, int);     // errore: + built-in
```

Overloading: helper function

- Pattern di operatori implementati preferibilmente con helper function
  - operator+, operator-, operator*, operator/: due argomenti
    - Almeno uno UDT, l'altro può essere anche built-in
- **Helper function è obbligatoria per << e >>**
  - operator <<: un argomento ostream&, ritorna ostream&
  - operator>>: un argomento istream&, ritorna istream&

Overloading operatore <<

```
vector<string> month_tbl;
// month_tbl inizializzato con le stringhe dei nomi dei mesi

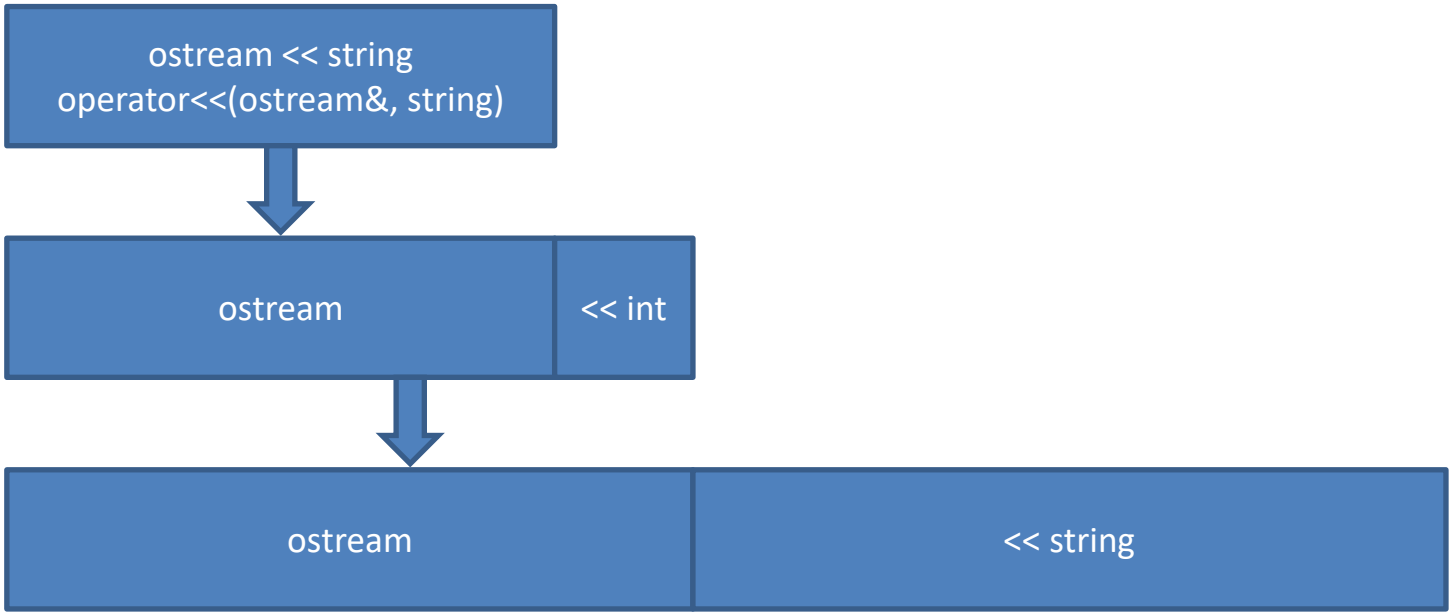
ostream& operator<<(ostream& os, Month& m)
{
    return os << month_tbl[int(m)];
}
```

- operator<< deve ritornare lo stream passato in input

Overloading operatore <<

```
int i = 4;

cout << "Il valore " << i << " è stato scritto in i\n";
```



Overloading operatore ++

- operator++ ha due implementazioni
  - preincremento
  - postincremento

```
T& operator++(T& t);          // preincremento
T operator++(T& t, int);      // postincremento: argomento
                              // int dummy
```

- Postincremento ritorna una copia del valore precedente (non è lvalue)
  - Può ritornare T o T&
  - Più comunemente ritorna T

Overloading operatore ++

```
Month operator++(Month& m)
{
    m = (m == Month::dec) ? Month::jan : Month(int(m) + 1);

    return m;
}
```

- operator++ può ritornare Month o Month&
  - Il comportamento è diverso!

Operatore ternario

```
Month operator++(Month& m)
{
    m = (m == Month::dec) ? Month::jan : Month(int(m) + 1);

    return m;
}
```

- Notare l'operatore "?:" – **operatore ternario**
  - m diventa jan se m == Month::dec, altrimenti Month(int(m) + 1)

Operatore ternario

- :?
- <condizione> ? <se vero> : <se falso>
- Esempio:

```
int a = 0, b = 4;

int max = a > b ? a : b;
```

- Utile per scritture più compatte
  - Quando le espressioni non sono troppo lunghe



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Overloading degli operatori

Stefano Ghidoni

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE